Exp: 1

17/7/2024

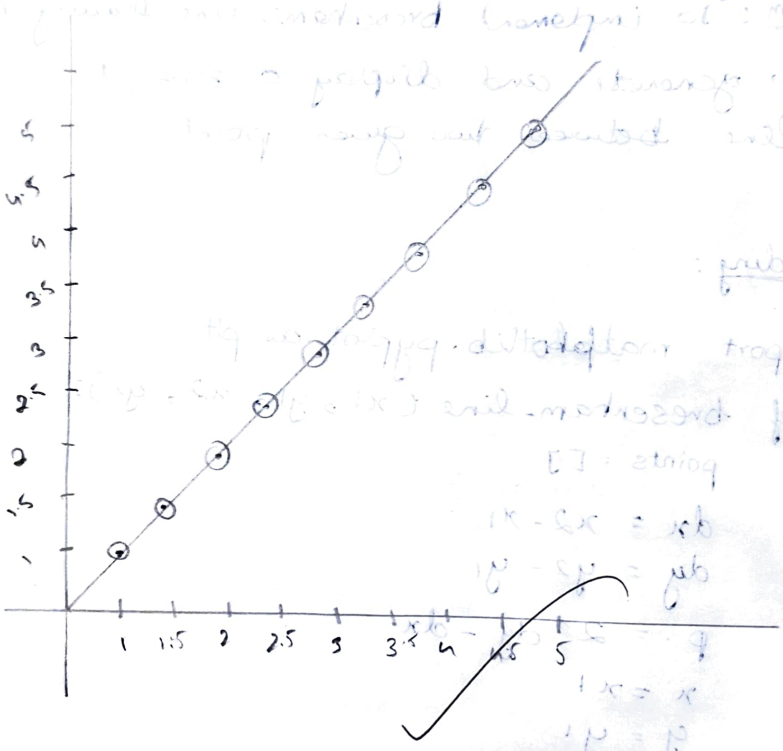
BRESENHAM LINE DRAWING

Aim: To implement bresenham's Line Drawing Algorithm to generate and display a straight line between two given point

Coding:

```
import matplotlib.pyplot as plt
def bresenham_line(x1, y1, x2, y2):
    points = []
    dx = x2 - x1
    dy = y2 - y1
    p = 2 * dy - dx
    x = x1
    y = y1
    points.append((x, y))
    while x < x2:
        x = x + 1
        if p < 0:
            p = p + 2 * dy
        else:
            y = y + 1
            p = p + 2 * dy - 2 * dx
        points.append((x, y))
    return points
x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
```

x2 = int (input ("Enter x2 : "))



```
x2 = int (input ("Enter x2: "))
y2 = int (input ("Enter y2: "))
points = bresenham_line (x1, y1, x2, y2)
print ("Points generated: ")
for p in points:
    print (p)

x_values = [p[0] for p in points]
y_values = [p[1] for p in points]

plt.plot (x_values, y_values, marker = "o")
plt.title ("Bresenham Line Drawing Algorithm
            (|m| < 1)")
plt.xlabel ("X-axis")
plt.ylabel ("Y-axis")
plt.grid ()
plt.show ()
```

23/7/26

Result:

The bresenhams line drawing was successfully executed.

Exp: 2 17/7/26

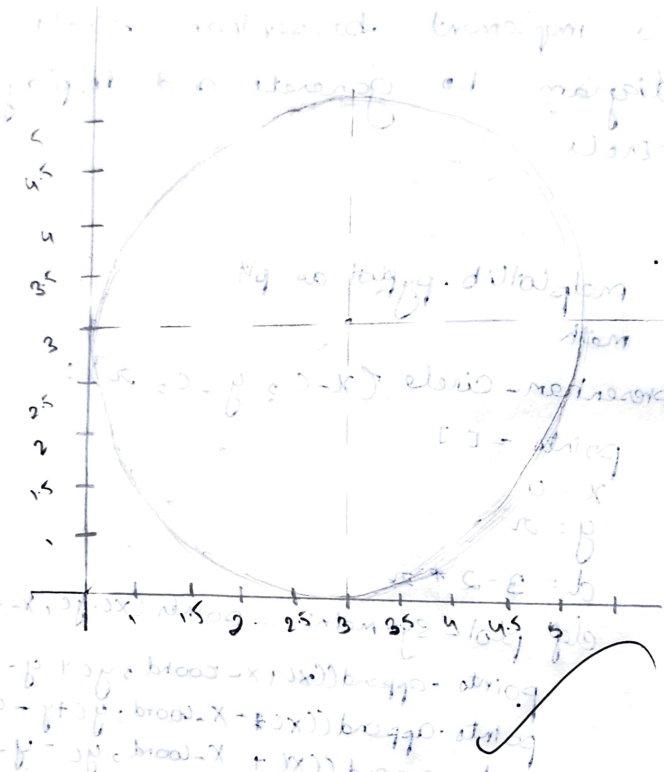
Bresenham Circle Diagram

Aim:

To implement bresenham circle diagram to generate and display circle

Code:

```
import matplotlib.pyplot as plt
import math
def bresenham_circle(x_c, y_c, r):
    points = []
    x = 0
    y = r
    d = 3 - 2 * r
    def plot_symmetric_points(xc, yc, x_coord, y_coord):
        points.append((xc + x_coord, yc + y_coord))
        points.append((xc - x_coord, yc + y_coord))
        points.append((xc + x_coord, yc - y_coord))
        points.append((xc - x_coord, yc - y_coord))
        points.append((xc + y_coord, yc - y_coord))
        points.append((xc + y_coord, yc + x_coord))
        points.append((xc - y_coord, yc + x_coord))
        points.append((xc + y_coord, yc + x_coord))
        points.append((xc - y_coord, yc - x_coord))
    plot_symmetric_points(x_c, y_c, x, y)
while y >= x:
    x += 1
    if d > 0:
        y -= 1
        d = d + 4 * (x - y) + 10
    else:
        d = d + 4 * x + 6
    plot_symmetric_points(x_c, y_c, x, y)
return points
```


4.5
4
3.5
3
2.5
2
1.5
1
1
1.5
2
2.5
3
3.5
4
4.5
5

```
x_center = int(input("Enter the x-coordinate of the center: "))
y_center = int(input("Enter the y-coordinate of the center: "))
radius = int(input("Enter the radius of the circle."))
raw_circle_points = bresenham_circle(x_center, y_center, radius)
unique_points = list(set(raw_circle_points))
def get_angle(point):
    px, py = point
    return math.atan2(py - y_center, px - x_center)
sorted_points = sorted(unique_points, key = get_angle)
x_coords = [p[0] for p in sorted_points]
y_coords = [p[1] for p in sorted_points]
if sorted_points:
    x_coords.append(sorted_points[0][0])
    y_coords.append(sorted_point[0][1])
plt.figure(figsize = (6,6))
plt.plot(x_coords, y_coords, '-')
plt.title("Bresenham Circle Drawing Algorithm")
plt.xlabel("X-axis")
plt.grid(True)
plt.gca().set_aspect('equal', adjustable='box')
plt.show()
```

Result: The Bresenham circle drawing was successfully executed.



Exp: 3

17/7/22

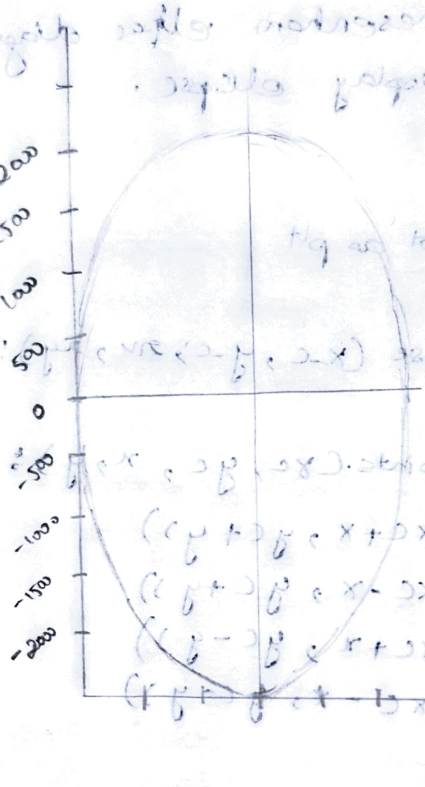
Bresenham Ellipse Diagram

Aim: To implement bresenham ellipse diagram to generate and display ellipse.

Code:

```
import matplotlib.pyplot as plt
import math
def bresenham_ellipse(x_c, y_c, rx, ry):
    points = []
def plot_symmetric_points(xc, yc, x, y):
    points.append((xc+x, yc+y))
    points.append((xc-x, yc+y))
    points.append((xc+x, yc-y))
    points.append((xc-x, yc-y))

rx_sq = rx * rx
ry_sq = ry * ry
x = 0
y = ry
p1 = round(ry_sq - (rx_sq * ry) + (0.25 * rx_sq))
dx = 2 * ry_sq * x
dy = 2 * rx_sq * y
plot_symmetric_points(x_c, y_c, x, y)
while dx < dy:
        x += 1
        dx += 2 * ry_sq
        if p1 < 0:
            p1 += dx + ry_sq
        else:
            y -= 1
            dy -= 2 * rx_sq
            p1 += dx - dy + ry_sq
        plot_symmetric_points(x_c, y_c, x, y)
```


200
150
100
50
0
-50
-100
-150
-200

```
p2 = round (ry_sq * (x+0.5) ** 2 + rx_sq * (y-1)**2
        - rx_sq * ry_sq)
while y >= 0:
    y -= 1
    dy -= 2 * rx_sq
    if p2 > 0:
        p2 += rx_sq - dy
    else:
        x += 1
        dx += 2 * ry_sq
        p2 += dx - dy + rx_sq
    plot_symmetric_points(x_c, y_c, x, y)
return points
x_center = int(input("Enter the x-coordinate of the center: "))
y_center = int(input("Enter the y-coordinate of the center: "))
radius_x = int(input("Enter the x-radius (semi-major/minor axis): "))
radius_y = int(input("Enter the y-radius (semi-major/minor axis): "))
raw_ellipse_points = bresenham_ellipse(x_center, y_center,
                                       radius_x, radius_y)
unique_points = list(set(raw_ellipse_points))
def get_angle(radius):
    px, py = radius
    return math.atan2(py - y_center, px - x_center)
sorted_points = sorted(unique_points, key=get_angle)
    plt.figure(figsize=(6,6))
    plt.plot(x_coords, y_coords, '-')
    plt.title("Bresenham Ellipse drawing Algorithm")
    plt.xlabel("x-axis")
    plt.ylabel("y-axis")
    plt.grid(True)
    plt.gca().set_aspect('equal', adjustable='box')
    plt.show()
```

Result:

Thus the bresenham ellipse drawing was successfully implemented.

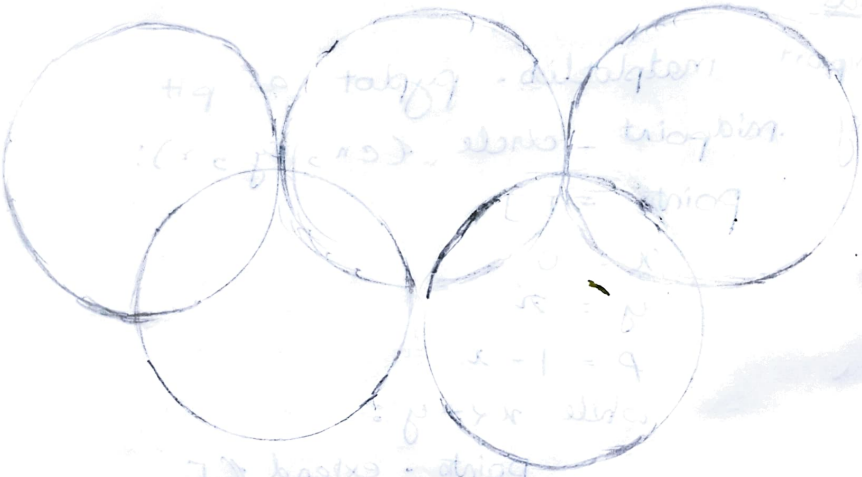
Exp: 4 2(a) 24/7/26

Draw the olympic logo using Ellipse (Mid point Algorithm)

Aim: To draw the olympic logo using ellipse & mid-point algorithm

Code:

```
import matplotlib.pyplot as plt
def midpoint_circle(cx, cy, r):
    points = []
    x = 0
    y = r
    p = 1 - r
    while x <= y:
        points.extend([
            (cx + x, cy + y),
            (cx - x, cy + y),
            (cx + x, cy - y),
            (cx - x, cy - y),
            (cx + y, cy + x),
            (cx - y, cy + x),
            (cx + y, cy - x),
            (cx - y, cy - cy - x)
        ])
        if p < 0:
            p = p + 2 * x + 3
        else:
            p = p + 2 * (x - y) + 7
            y = 1
        x + = 1
    return points
```



```
def draw_circle(cx, cy, r):
    points = midpoint_circle(cx, cy, r)
    points.sort(key=lambda p: __import__
        ('math').atan2(p[1]-cy, p[0]-cx))
    x = [p[0] for p in points]
    y = [p[1] for p in points]
    plt.plot(x, y, linewidth=5)

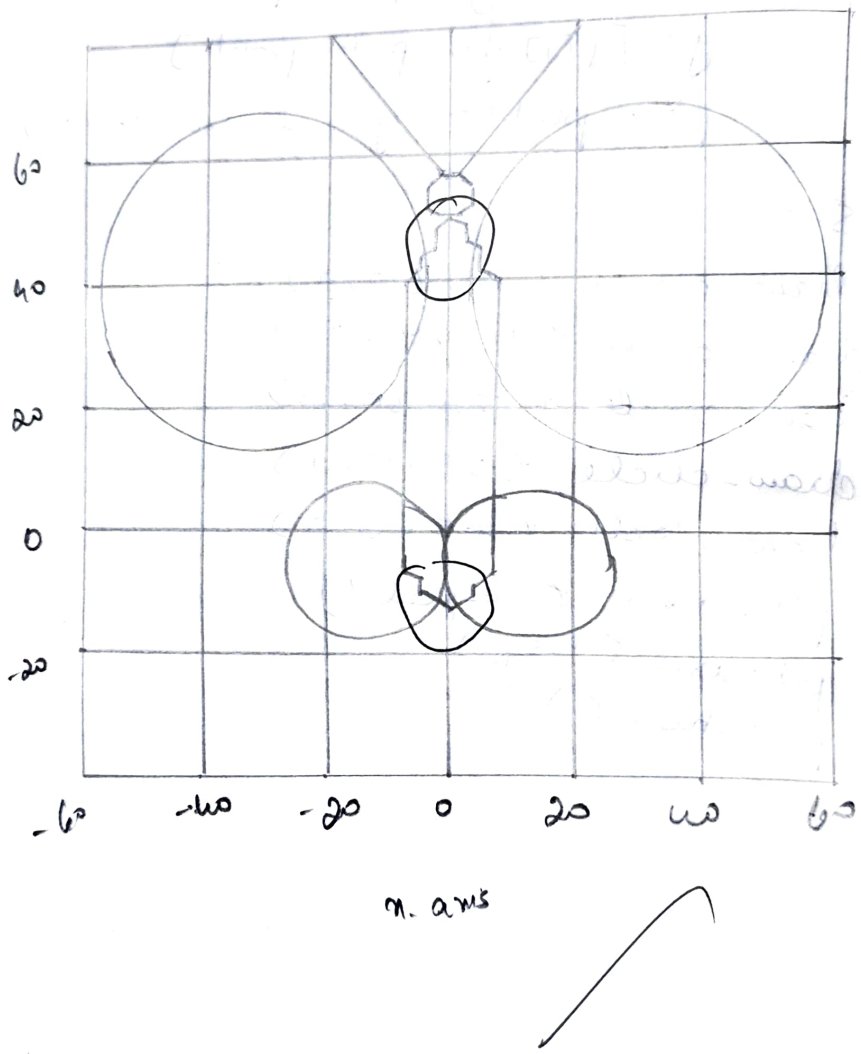
S=5.0
draw_circle(0, 60, r)
draw_circle(110, 60, r)
draw_circle(220, 60, r)
draw_circle(55, 0, r)
draw_circle(165, 0, r)
plt.axis("equal")
plt.axis("off")
plt.show()
```



Result:

The olympic logo using the mid point algorithm is ~~drawing~~ drawn successfully.

Butterfly using multiple ellipse



2(b) Exp: 5 24/7/26

Draw Butterfly using Midpoint Algorithm.

Aim:

To draw a butterfly using midpoint algorithm

Code:

```
import numpy as np, matplotlib as plt
def get_circle (xc, yc, r):
while x < = y :
    for dx, dy in
        pts . append (cx + dx + yc + dy)
    Piy = (pt 2 * x + 3 * y)
    x + = 1
return pts
def plot . fill (pts . xc, yc, color)
    plt = sorted (set (pts), key = lambda
        p: np as tan2 (p[1] - yc, p[0] - xc))
    plt . figure (fig size . (6, 6))
    plot - fill (get - ellipse (2, 6, 4, 3) 'Lot pink'
    plot : fill (get_circle (5, 6, 1) 'black')
    plot - fill (get_circle (5, 9, 4) 'black')
    plt . axis ('equal')
    plt . grid (True)
    plt . show
```

Result:

Thus a butterfly is drawn successfully.

40
30
20
10
0
-10
-20

Exp 6 31/7/24

2(c) Draw smiley face using midpoint circle algorithm.

Aim:

To draw a smily face using mid point circle algorithm.

Code:

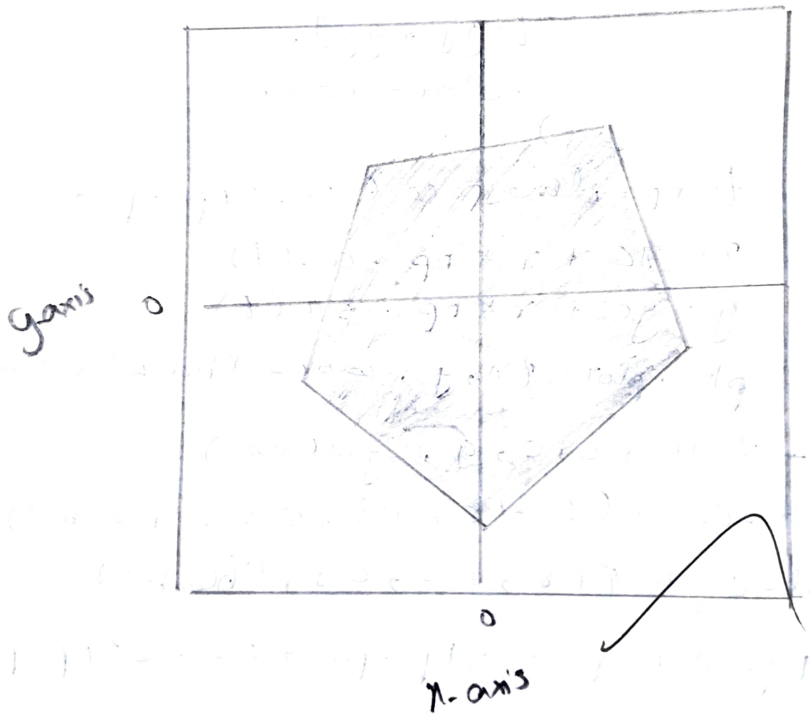
```
import matplotlib.pyplot as plt
import numpy as np
def mid_point_circle(xc, yc, r):
    points = []
    x = 0
    y = r
    p = 1-r
    while x <= y:
        points.extends([
            (xc+x, yc+y)
            (xc-x, yc+y),
            (xc+x, yc-y),
            (xc-x, yc-y),
            (xc+y, yc+x),
            (xc+y, yc+x),
            (xc-y, yc+x),
            (xc+y, yc-x),
            (xc-y, yc-x)
        ])
        if p<0:
            p+= 2*x+3
        else:
            x+=1
    return points
```

```
filled_circle(xc, yc, r, color)
    y = np.arange(yc-r, yc+r, 0.01)
    for yy in y:
        half = np.sqrt(r*r - (yy-yc)**2)
        plt.plot(
            [xc-half, xc+half],
            [yy, yy],
            color = color
        )
    t = np.linspace(0, 2*np.pi, 300)
    x = xc + r*np.cos(t)
    y = yc + r*np.sin(t)
    plt.plot(x, y, color = "black", linewidth=9)
filled_circle(0, 0, 9, "yellow")
filled_circle(0-1.8, 1.5, 0.5, "black")
filled_circle(1.8, 1.5, 0.5, "black")
t = np.linspace(np.pi+0.2, 2*np.pi-0.2, 15)
x = 2.5 * np.cos(t)
y = 1.5 * np.sin(t) - 0.5
plt.show()
```

Result:

To draw smiley face using midpoint algorithm is done successfully.

Polygon using flood fill algorithm



Aim:

To draw a polygon using flood fill algorithm

Code:

```
import matplotlib.pyplot as plt
import numpy as plt
width = 300
height = 800
image = np.ones(height, width, 3),
        dtype = np.(uint8) * 255
polygon = [(150, 50), (250, 130), (210, 340),
           (90, 240), (10, 130)]
def draw_line(x1, y1, x2, y2):
    points = max(abs(x2 - x1), abs(y2 - y1))
    for i in range(points + 1):
        x = int(x1 + (x2 - x1) * i / points)
        y = int(y1 + (y2 - y1) * i / points)
        image[y, x] = [0, 0, 0]
for i in range(len(polygon)):
    x1, y1 = polygon[i]
    x2, y2 = polygon[(i+1) % len(polygon)]
    draw_line(x1, y1, x2, y2)
def flood_fill(x, y, old_color, new_color):
    stack [(x, y)]
while stack:
    x, y = stack.pop()
    if x<0 or x>= width or y <0 or
                y>= height continues
```

if not np.array_equal(image[y, x],
old_color):

continue

image[y, x] = new_color

stack.append((x+1, y))

stack.append((x-1, y))

stack.append((x, y+1))

stack.append((x, y-1))

floodFill(

150, 150,

np.array([255, 255, 255], dtype=np.uint8),

np.array([0, 180, 0], dtype=np.uint8))

plt.imshow(image)

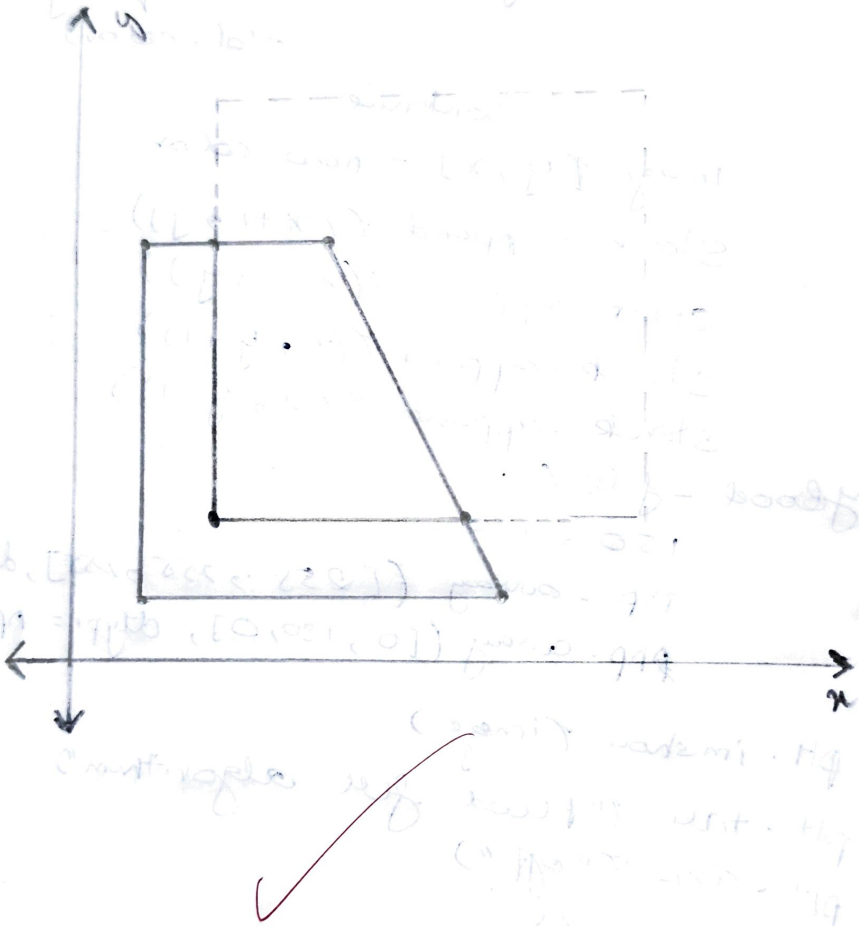
plt.title("Flood fill algorithm")

plt.axis("off")

plt.show()

Result:

A polygon is drawn using the flood fill algorithm successfully.


y
x

Exp:8 14/8/26

Sutherland - Hodgman polygon clipping algorithm

Aim:

To implemat the algorithm to clip a subject polygon against a rectangular clipping window using python.

program:

```
import numpy as np
import matplotlib.pyplot as plt.
def inside ( p, a, b):
    return np.cross (b-a , p-a)>= 0
    def interest (p1, p2 , a, b):
        d1 = p2 - p1
        d2 = b - a
        c = np.cross (a - p1 ,d2) /np.cross (d1 , d2 )
        return p1 + c * d1
def clip (subject , window):
    output = list (subject)
for i in range (len ( window )) - 1) :
    a, b = window [i] , window [i+1]
    inp , output = output ,[]
for p in range (len ( window )) -1):
    p1 , p2 = inp [j-1] , inp [j]
    in1 , in2 = inside (p1 , a , b), inside (p2 ,a, b)
if . in 2 :
    if not in1 :
        output , append ( intersect (p1 , p2 , a , b ))
elif in 1 :
    output . append (intersect (p1 , p2 , a, b))
return np. array (output)
```

```
subject = np.array([[0.5, 0.5], [3, 0.5], [2, 3], [0.5, 3], [0.5, 0.5]])
window = np.array([[1, 1], [4, 1], [4, 4], [1, 4], [1, 1]])
result = clip(subject, window)
plt.plot([subject[i, 0] for i in range(len(subject))], [subject[i, 1] for i in range(len(subject))], 'b-o', label='subject')
if len(result) > 0:
    result = np.vstack([result, result[0]])
plt.title("polygon clipping"), plt.xlabel('x'), plt.ylabel('y')
plt.legend()
plt.show()
```

Result:

The program for the sutherland - hodgman polygon clipping algorithm is done.

Exp: 9 19/8/26

Implementation scan conversion of polygon and use flood fill algorithm

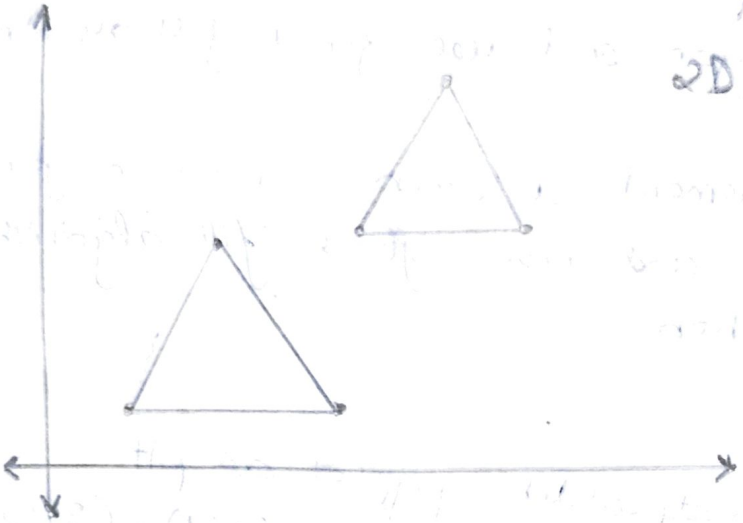
Aim: To implement a scan conversion for a polygon and use flood fill algorithm in python.

Python:

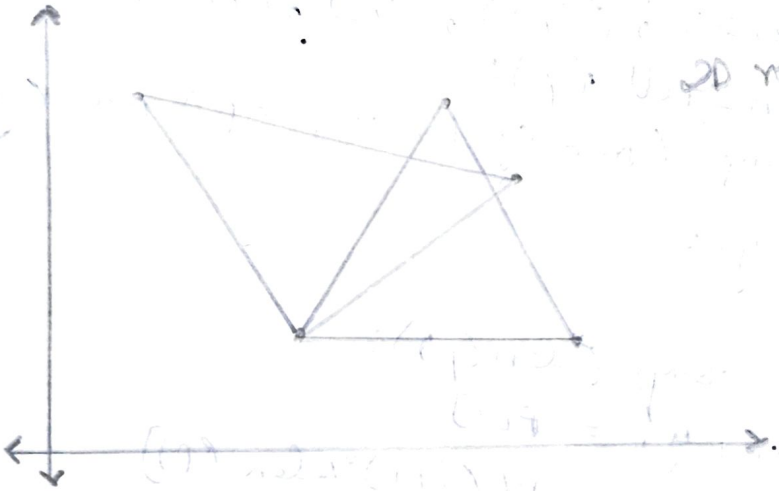
```
import matplotlib.pyplot as plt
p = [(2,2),(5,8),(9,3),(7,1),(3,4)]
def scan_fill(p):
ys = range(min(y for x,y in p), max(y for x,y in p)+1)
for y in ys:
    x = []
    for i in range(len(p)):
        x1, y1 = p[i]
        x2, y2 = p[(i+1)%len(p)]
        if y1 != y2 and min(y1,y2) <= y < max(y1,y2):
            x.append(x1+(y-y1)*(x2-x1)/(y2-y1))
    x.sort()
    for i in range(0,len(x),2):
        if i+1 < len(x):
            plt.plot(x[i:i+2],[y,y],'b-')
x,y = x,y,p*(p+p[0]))
plt.plot(x,y,'k-',linewidth=2)
scan_fill(p)
plt.fill(x,y,colour='orange',alpha=0.7)
plt.axis('equal')
plt.title("Scan conversion and flood fill")
plt.show()
```

Result: The program for the scan conversion of polygon and use flood fill algorithm is done successfully

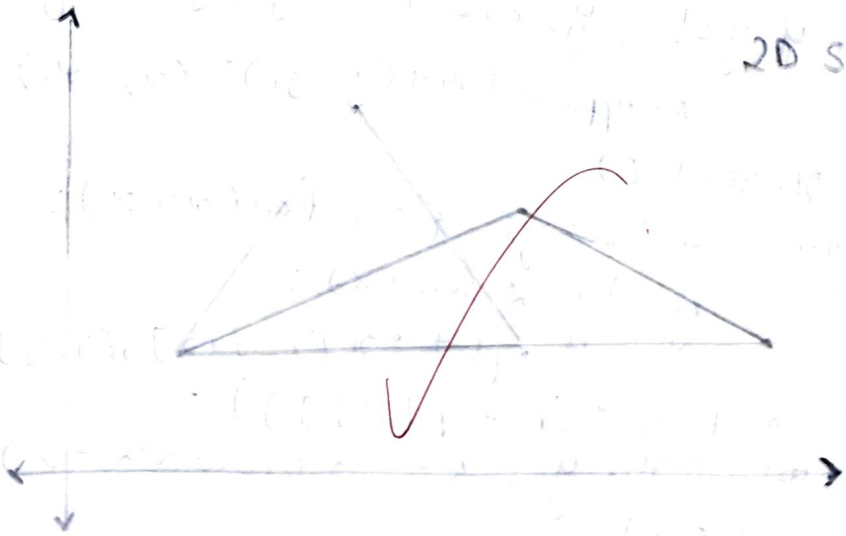
2D translation



2D rotation



2D scaling



Exp. 10

2/8/20

Basic Transformation for 2D shapes for triangle

Aim: To implement the program for the basic transformations for 2D shapes to draw triangle in python.

Program:

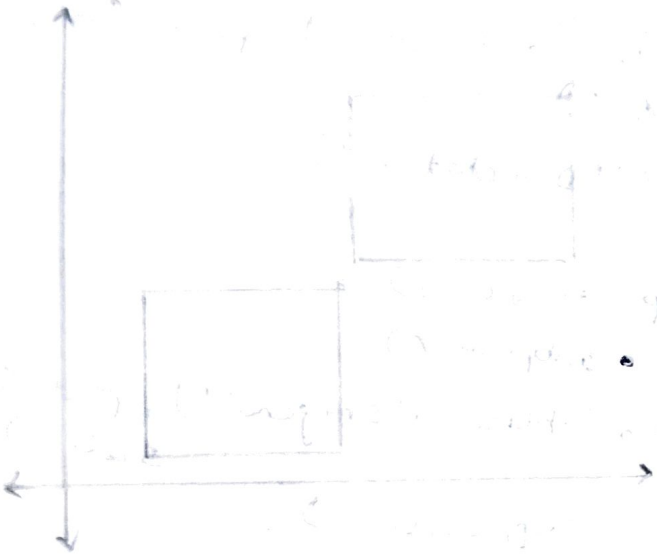
```
import numpy as np
import matplotlib.pyplot as plt
shape = np.array([[0,0],[2,0],[1,2],[0,0]])
def plot(ax, s, color, label):
    ax.plot(s[:,0], s[:,1], color = color,
            marker = 'o', label)
    ax.fill(s[i,0], s[s:1, 1], color = color, alpha=0.3)
    ax.grid()
    ax.axis('equal')
    ax.legend()
def translate(s, dx, dy):
    return s + [dx, dy]
def rotate(s, angle):
    a = np.radians(angle), R = np.array([[np.cos(a),
        -np.sin(a)],[np.sin(a), np.cos(a)])
return s @ R, T
def scale(s, sx, sy):
    return s*[sx, sy]
i) t = translate(shape, 45)
   fig, ax = plt.subplots()
   plot(ax, shape, 'blue', 'original'), plot(ax, t, 'red',
        'Translation')
   ax.set_title('2D Translation')
   plt.show()
```

ii) r = rotate (shape, 45)
fig, ax = plt.subplots()
plot (ax, r, 'blue', 'original'), plot (ax, r, 'green', 'rotated 45')
ax.set_title ('2D rotation')
plt.show()

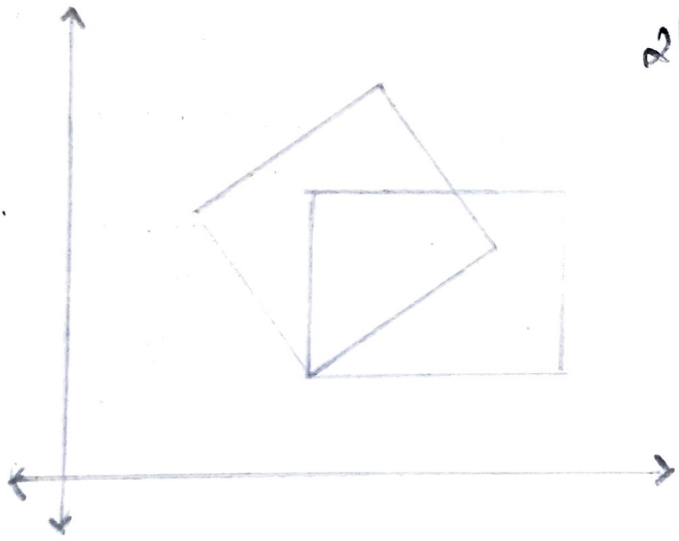
iii) S = scale (shape, 2, 0.5)
fig, ax = plt.subplots()
plot (ax, shape, 'blue', 'original'), plot (ax, S, 'purple', 'scale')
ax.set_title ('2D scaling')
plt.show()

Result: The program to draw the triangle for the basic transformations in python.

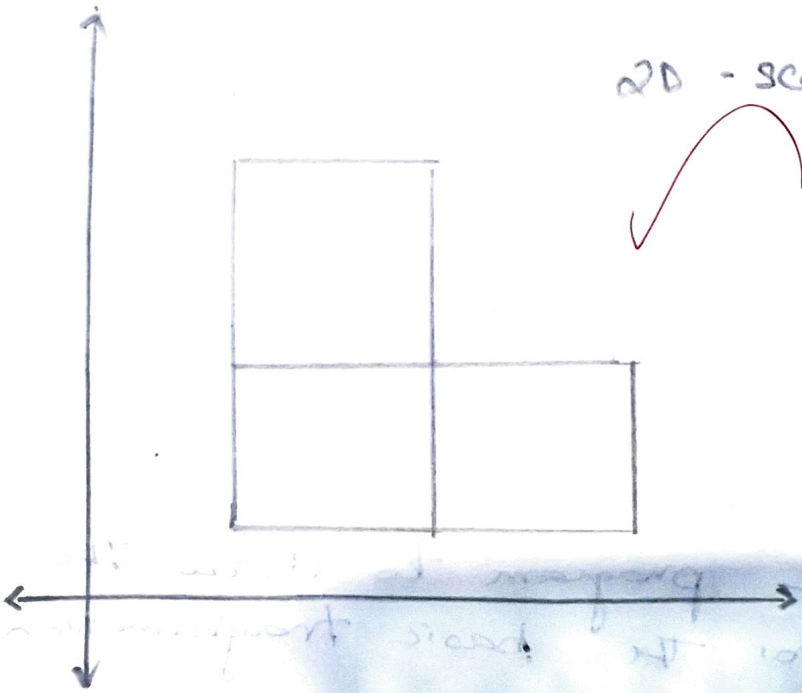
2D translation



2D rotation



2D - scaling



Basic Transformation for 2 shapes for rectangle

Aim: To implement the basic transformations for 2D shapes to draw rectangle in python.

Program:

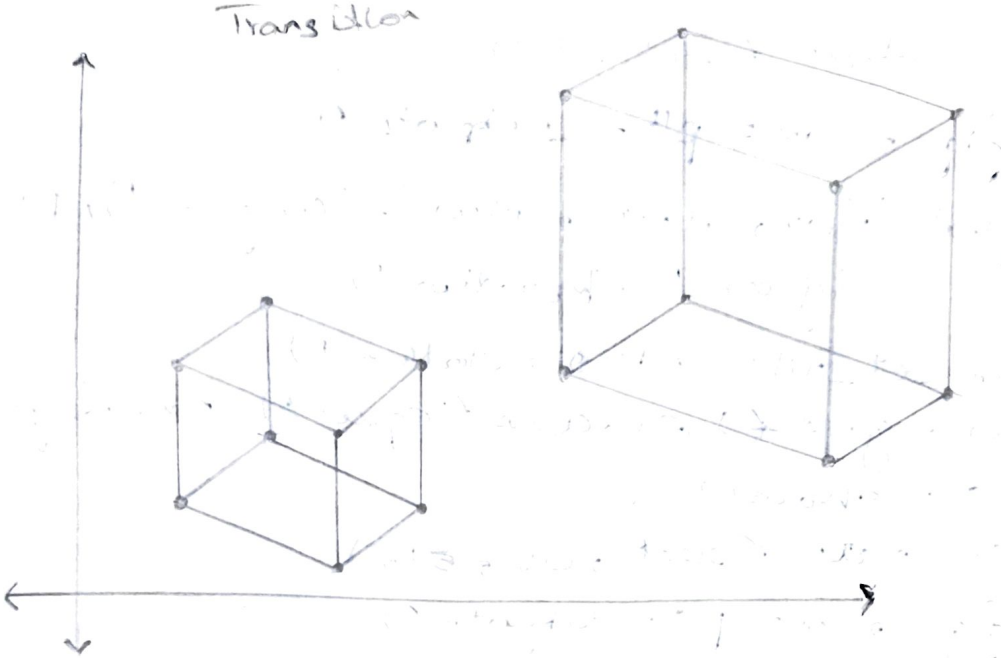
```
import numpy as np.
import matplot-pyplot as plt
def translate (s, dx, dy):
    return s + [dx, dy]
def rotate (s, angle):
    a = np.radians (angle)
    R = [[np, cos(a), - np, sin(a)],
         [np, sin(a), np. cos(a)]]
    return s @ np, array (R), T
def scale (s, sx, sy):
    return s = [sx, sy]
def plot (ax, s, color, label):
    ax, plot (s[:,0], s[:,1], color = color,
              label = label)
rect = np. array ([[0,0], [3,0], [3,2],
                   [0,2], [0,0])
t = translate (rect, 1, 2.5)
fig, ax = plt. subplots ()
plot (ax, rect, 'blue', 'original'), plot (ax, r,
    'red', 'Translation')
ax. set-title ('2D translation')
ax. grid ()  ax. axis ('equal') ax. legend ()
plt. show ()
```

```
r = rotate (rect, 30)
fig, ax = plt.subplots ()
plot (ax, rect, 'blue', 'Original'), plot (ax, r,
      'green', 'Rotation')
ax.set_title ('3D rotation')
ax.grid () ax.axis ('equal') ax.legend ()
plt.show ()
s = Scale (rect, 0.5, 1.5)
fig, ax = plt.subplots ()
plot (ax, rect, 'blue', 'Original'), plot (ax, s,
    'purple', 'scaling')
ax.set_title ('2D scaling')
ax.grid () ax.axis ('equal') ax.legend ()
plt.show ()
```

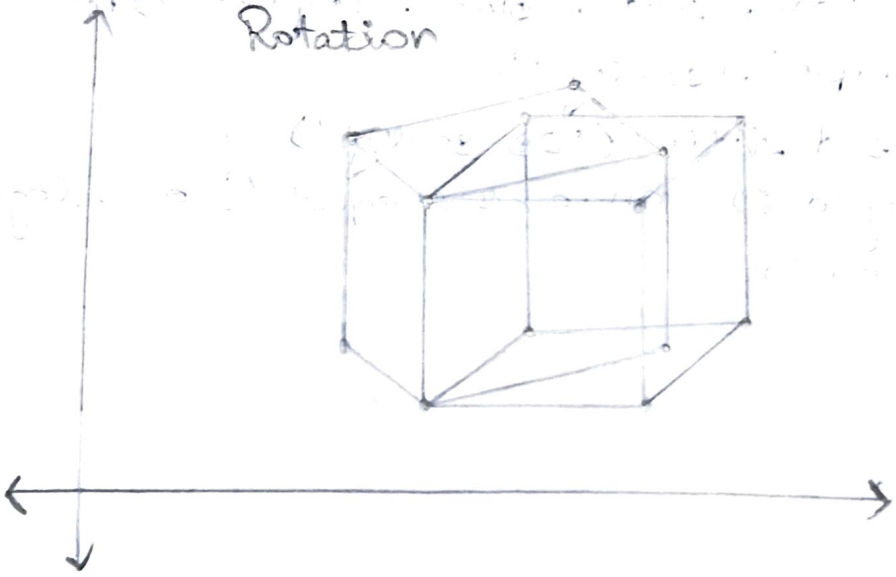
Result:

The program to draw the rectangle for the basic transformation in python has been executed successfully.

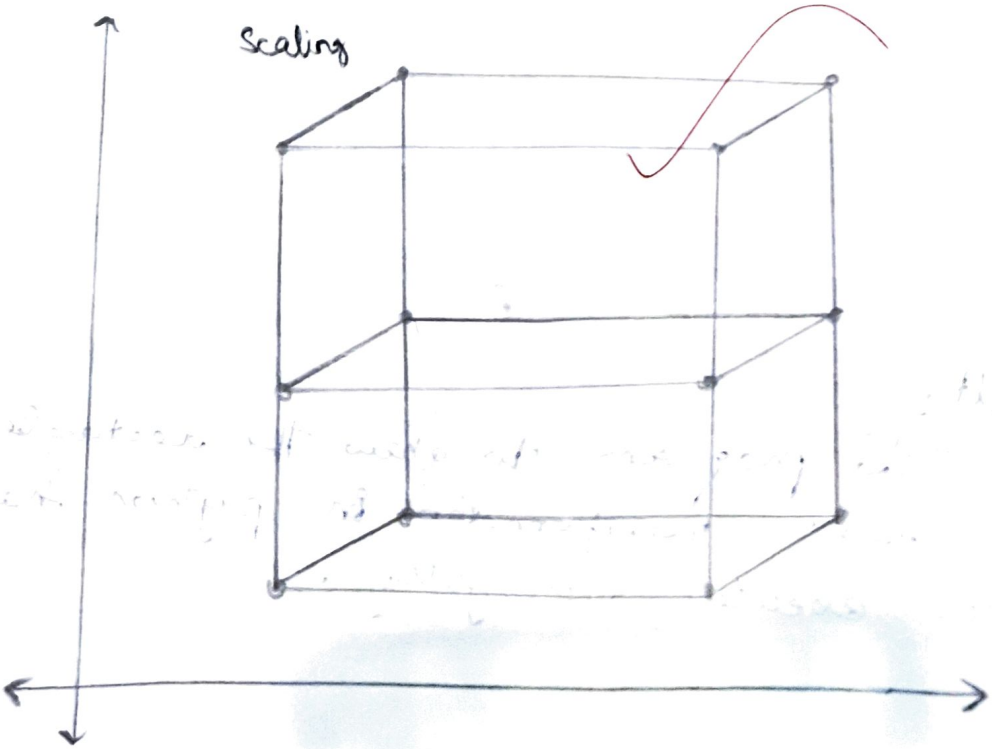
Translation



Rotation



Scaling



3D shapes Transformations

for Cube

Aim: To implement the transformations for 3D shapes for cube using the python.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import poly3d collection
v= np.array([[0,0,0],[1,0,0],[0,1,0],[0,0,1],[1,1,0],
             [1,0,1],[0,1,1],[1,1,1]])
H = np.c_[v, np.ones(8)]
F = [(0,1),(0,2),(0,3),(1,4),(1,5),(2,4),(2,6),(3,5),
     (3,6),(4,7),(5,7),(6,7)]
F = [(0,1,4,2),(3,5,7,6),(0,1,5,3),(2,4,7,6),
     (0,2,6,3),(1,4,7,5)]
def plot(T, title, color):
w = (T @ H.T)[:,:3]
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')
for v,c in [(v,'blue'),(w,color)]:
    for i,j in F:
        ax.plot(*zip(v[i],v[j]), color=c)
    ax.scatter(v[:,0],v[:,1],v[:,2],color=c)
    faces = [[v[k] for k in f] for f in F]
    ax.add_collection3d(Poly3DCollection(
        faces, alpha=.2, facecolor=c))
ax.set(xlabel='x', ylabel='y', zlabel='z',
       title=title)
plt.show()
```

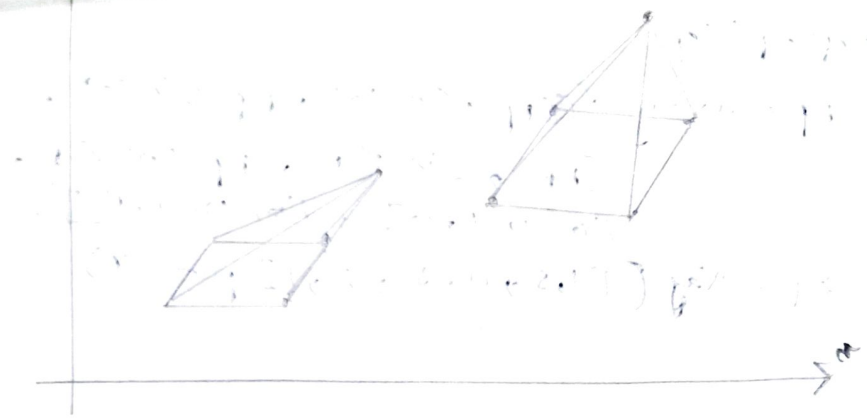
```
T = np.array([[1,0,0,2],[0,1,0,2],[0,0,1,2],
              [0,0,0,1]])

a = np.pi/4
T = np.array([[np.cos(a), np.sin(a), 0, 0],
              [np.sin(a), np.cos(a), 0, 0],
              [0, 0, 1, 0], [0, 0, 0, 1]])
T = np.diag([1.5, 0.5, 2, 1]) plot(T)
```

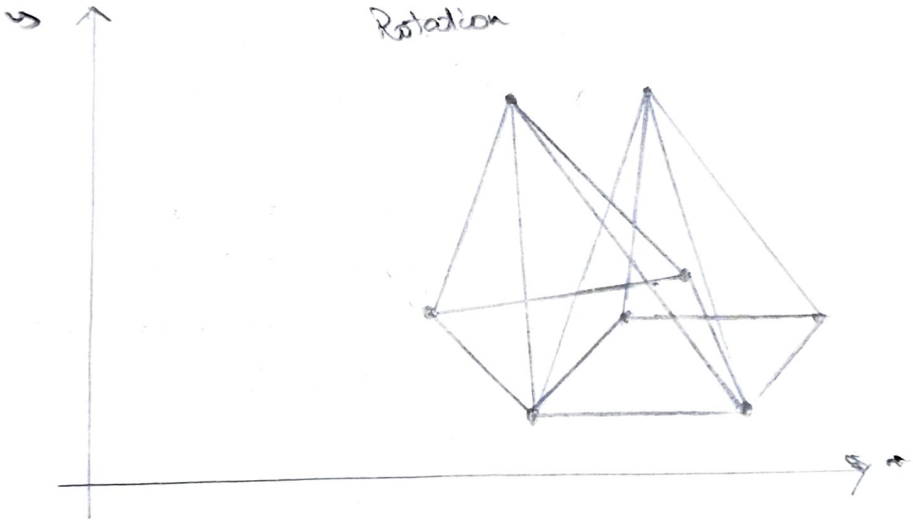
Result:

The program for the 3D transformations for cube is done.

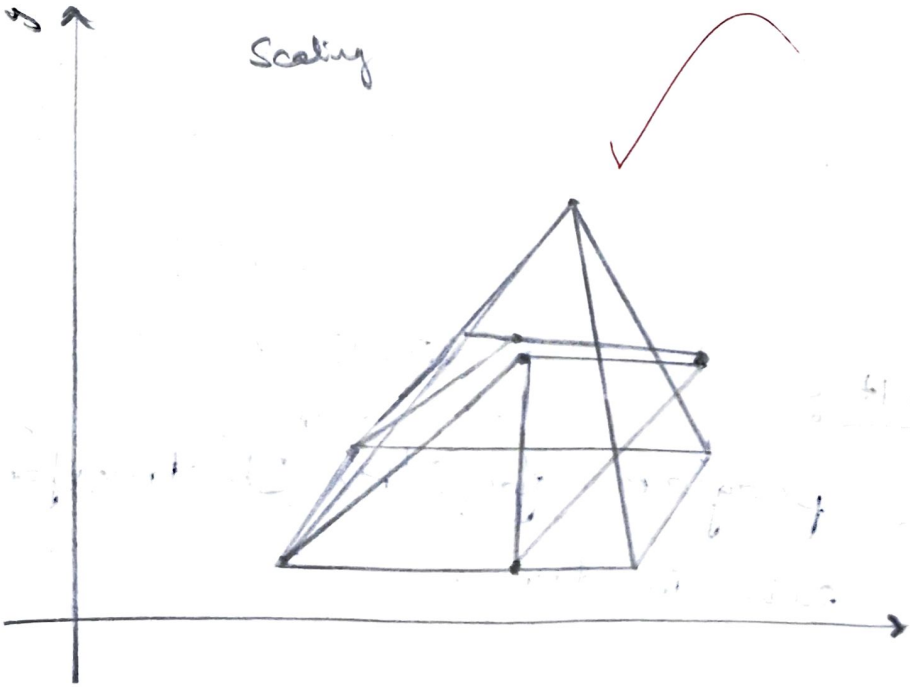
Translation



Rotation



Scaling



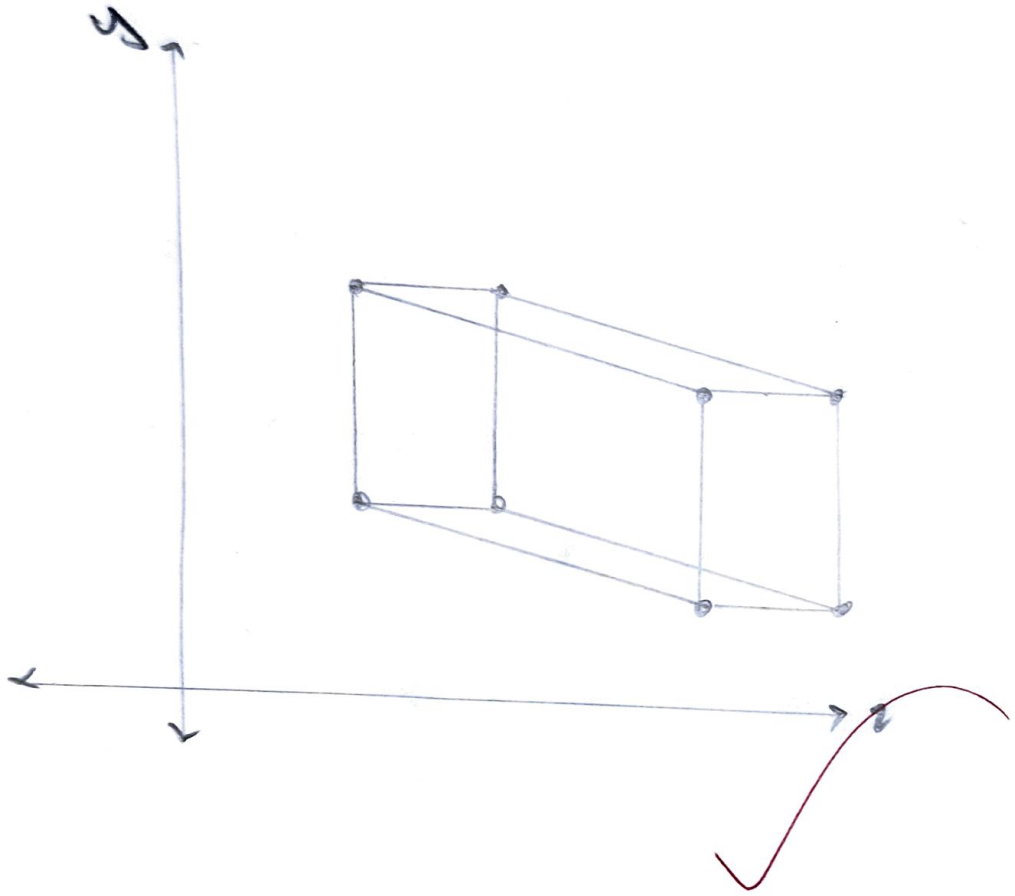
Aim: To implement the program for the 3D shapes transformation for cone.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
V = np.array([[0,0,0], [1,0,0], [1,1,0], [0,1,0],
              [.5,.5,1]])
H = np.c_[V, np.ones(5)]
E = [(0,1), (1,2), (2,3), (3,0), (0,4), (1,4), (2,4),
     (3,4)]
F = [[0,1,2,3], [0,1,4], [1,2,4], [2,3,4], [2,3,4],
                                    [3,0,4]]
def plot(T, title, color):
    w = (T @ H.T).T[:, :3]
    fig = plt.figure(figsize=(5,6))
    ax = fig.add_subplot(111, projection='3d')
for p, c in [(V, 'blue'), (w, colour)]:
    for i,j in E:
ax.add_collection3d(Poly3DCollection([[p[k]
                    for k in f] for f in F],
                    alpha=.2, facecolor=c))
ax.set(xlabel='x', ylabel='y', zlabel='z',
       title=title())
plot.show()
T = np.array([[1,0,0,2], [0,1,0,2], [0,0,1,2],
              [0,0,0,1]])
a = np.pi/4
```

```
T= np.array([[np.cos(a), -np.sin(a), 0, 0],
    [np.sin(a), np.cos(a), 0, 0], [0, 0, 1, 0],
        [0, 0, 0, 1]])
T= np.diag([1.5, .5, 2, 1])
```

Result : The program for 3D transformations for the cone is done implemented.


y
z

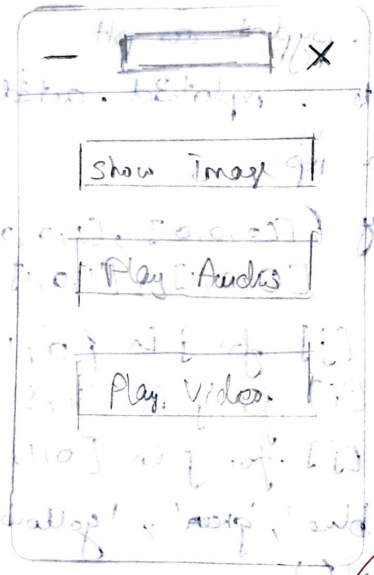
Using Polygons and shading techniques.

Aim: To create and rendered 3d objects using polygonal modelling and apply flat shading & basic color techniques

Program:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import numpy as np
vertices = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],
                     [0,0,1],[1,0,1],[1,1,1],[0,1,1]])
faces = [[vertices[j] for j in [0,1,2,3]],
         [vertices[i] for j in [4,5,6,7]],
         [vertices[j] for j in [0,1,5,4]]]
colors = ['red','blue','green','yellow','orange','purple']
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
cube = Poly3DCollection(faces, facecolors=colors, alpha=0.2)
ax.add_collection3d(cube)
ax.scatter(vertices[i,0], vertices[i,1], vertices[i,2],
           color='black')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.set_xlim(0,1)
ax.set_ylim(0,1)
plt.show()
```

Result: 3D polygon objects were successfully created and rendered.


Show Image
Play Audio
Play Video

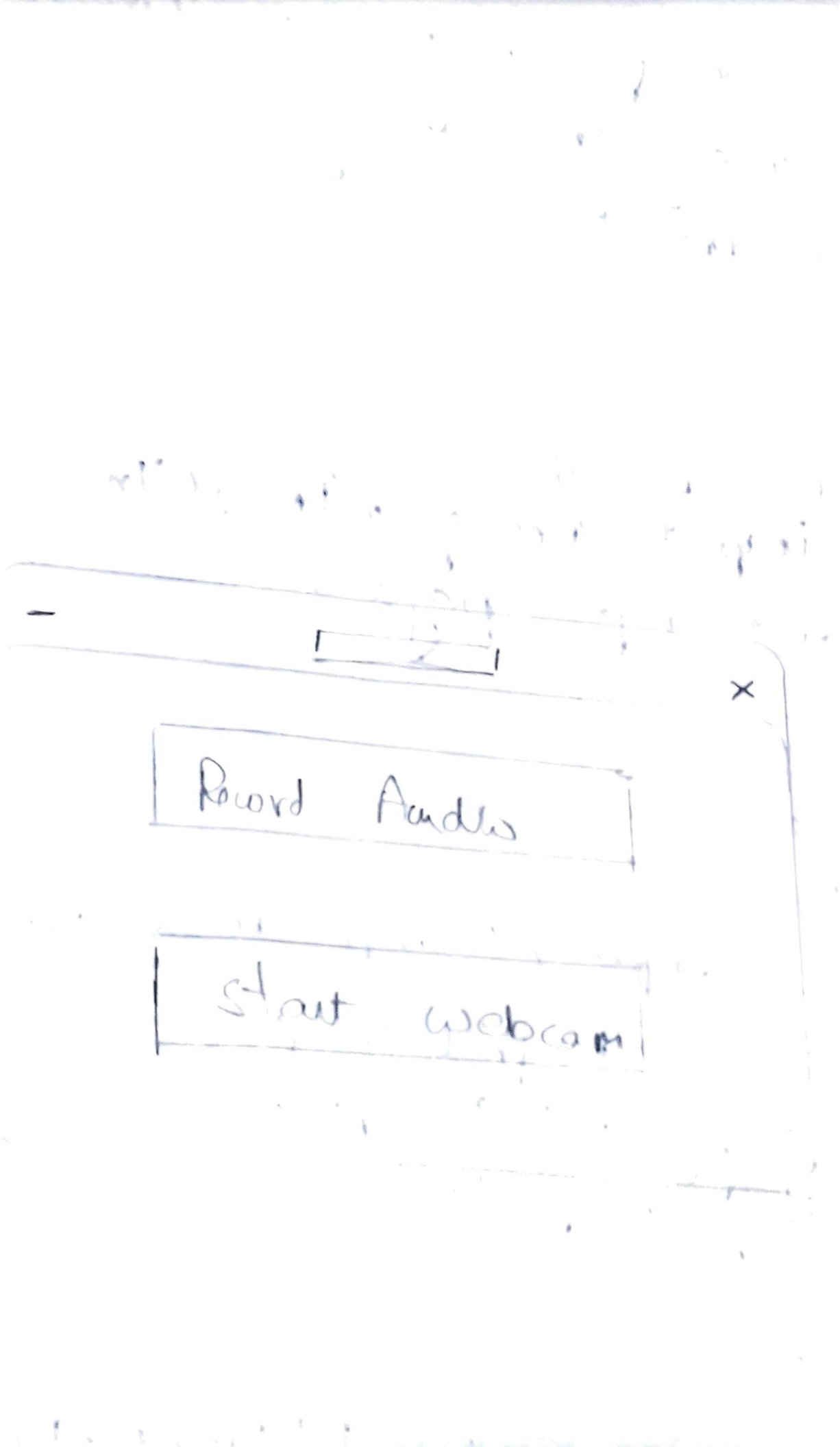
Aim: To develop a simple multimedia application that integrates images, video and audio in UI.

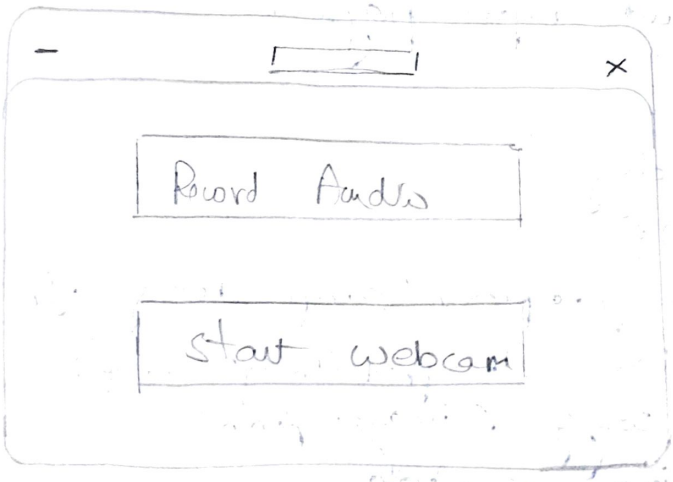
Program:

```
import tkinter as tk
from PIL import Image, ImageTk
from playsound import playsound
import cv2
import threading
def image():
    img = cv2.imread("sample_image.jpg").shape(500,300)
    photo = ImageTk.PhotoImage(img)
    panel.config(image=photo)
    panel.image = photo

def video():
    cap = cv2.VideoCapture("sample.mp4")
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow("Video", frame)
        if cv2.waitKey(25) & 0xFF == ord("q"):
            break
    cap.release()
    cv2.destroyAllWindows()
root.title("Multimedia App")
panel.pack()
tk.Button(root, text="Show Image", text="Play Audio",
          text="Play Video")
root.mainloop()
```

Result: A multimedia application that integrates images, video and audio in UI has been executed successfully.


Record Audio
Start Webcam


Record Audio
start webcam

Exp16 9/10/24

Capture video / Audio from webcam or microphone and display in multimedia

Aim: To develop a program that captures live video, audio from the webcam and microphone.

Program:

```
import tkinter as tk
import cv2
import sounddevice as sd
from spicy.io : wavfile import write
import time
def audio():
fs = 44100
print("Recording.....")
data = sd.rec(5*fs, completed = fs, channels=2)
sd.wait()
write("recorded_audio.wav", fs, data)
print("Audio saved!")
def video():
    cap = cv2.VideoCapture(0)
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow("webcam", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
    break
    cap.release()
    cv2.destroyAllWindows()
    root.title("Audio and ~~Video~~ Video Capture")
    tk.button(root, text = "Record video & stop webcam"
    root.mainloop()
```

Result: Live video, audio from the webcam and microphone captured successfully.